Project Report

Ride Sharing System

Submitted by:

Bismoy Sarker
Roll: 2407008

Avik Hasan
Roll: 2407021

Farhan Hossain
Roll: 2407019

Submitted to:

Subah Nawar
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Lamisa Bintee Mizan Deya
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

1. Project Glossary

This project is a comprehensive, console-based Ride-Sharing and Transportation Management System developed in C++ using the algorithms of basic data structure that has been taught over the last lab experiments. This project models real-world ride-hailing mechanics including driver-rider matching, dynamic surge pricing (based on weather, traffic, and demand), wallet management, and map routing. The project implemented all the necessary algorithms which is needed to develop any real life simulating system.

2. Core system modules

- ***Admin Dashboard:***

  Through it,one can have centralized control over the system. The admins can view earnings,manage pricing tiers(i.e Bikes,CNG,UberX),alter surge parameters(i.e traffic/weather multiplier) and approve wallet top up requests

- ***Driver management:***

  Through it, one can maintain their driver profile.These portal supports to view their profile &earnings ,Toggling their active status,looking up to their trip history and updating their current location

- ***User/Passenger interface:***

  This interface allows users or passenger to book their ride,show their latest trip history,adding money in their wallet or checking balance and adding their favourite locations, All these features leverages a user's riding experience

- ***Authentication Systems:***

  Secure login system is a pre requisite for any real life simulation model. So our project has a unique verification process for users,drivers and admins. It requires username and password which has to be unique otherwise the access is denied.

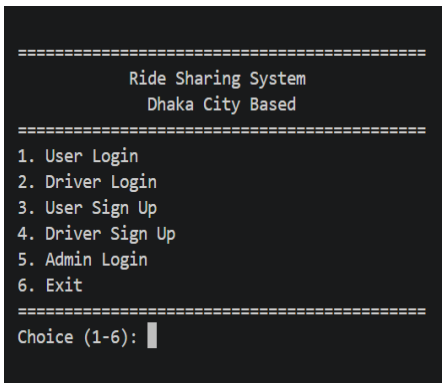


Fig 1:Main menu Interface

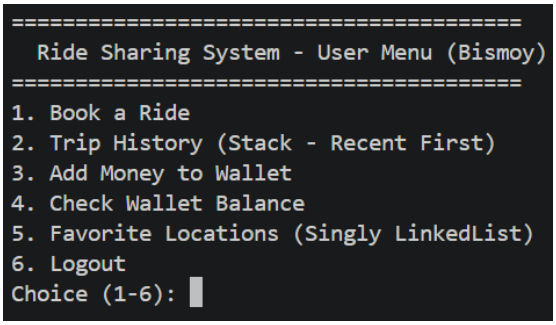


Fig 2:User menu Interface

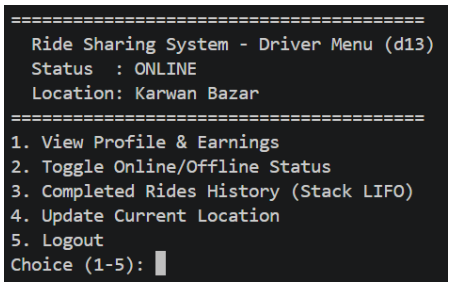


Fig 3:Driver menu Interface

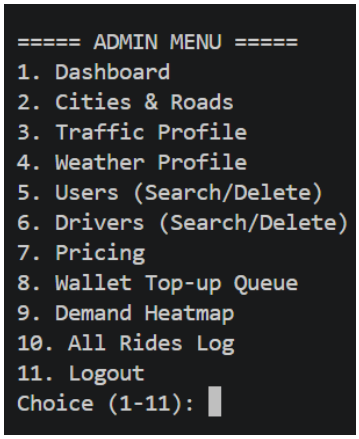


Fig 4:Driver menu Interface

3. Data Structure and Implementations

To increase the performance and efficiency of the system, different types of data structures and sorting algorithms are used. Also we have used self-made custom data structures that helps the system to perform on its vast data more easily and rapidly.

Data Structure	Implementation and file reference	System use Case
Binary Search tree(BST)	UserAccountBST(bst.cpp)	Used to store and search Rider profiles by ID. Enables fast O(log n) lookups, insertions, and deletions when the admin is querying millions of potential users.
Weighted Graph	CityGraph(graph.hpp)	Represents the city layout. Nodes represent cities, and weighted edges represent road distances. Used for routing and calculating base fare per kilometer.
Dynamic Arary	Vector<T>(vector.hpp)	A custom templated array that scales automatically. Used extensively across the platform to maintain drivers, ride logs, and leaderboard indices.
Stack	Stack<T>(stack.hpp)	Used internally within specific graph traversals and path tracking.
Queue	WalletRequestQueue(queue.cpp)	Used in admin interface and bfs traversal to store value temporarily
Linked list	SinglyLinkedList(linkedlist.hpp)	Used vastly in graphs,and riders's favourite location.
Min heap	matchDriver(heap.cpp)	Used in the function to determine the available drivers closest to the passenger's location.

4.Applied algorithms

- *Recursive Binary Search:*

  This searching algorithm used within”DriverModule” to execute lightning-fast searches against the driver dataset. The “searchDriverByRBS” function queries the array of DriverInfo objects

- *Merge sort:*

  The driver leaderboard ranks drivers based on earnings, completed trips, and ratings. A custom mergeSortDriverLeaderboard algorithm sorts the driver indices safely without copying large user objects in memory.

- *Selection Sort:*

  Used in generating the Demand Heatmap, The getTopDemandSelectionSort function parses historical trip data (rides.csv) .This calculates frequency per city and sorts them to display the most requested areas for both driver and passenger.

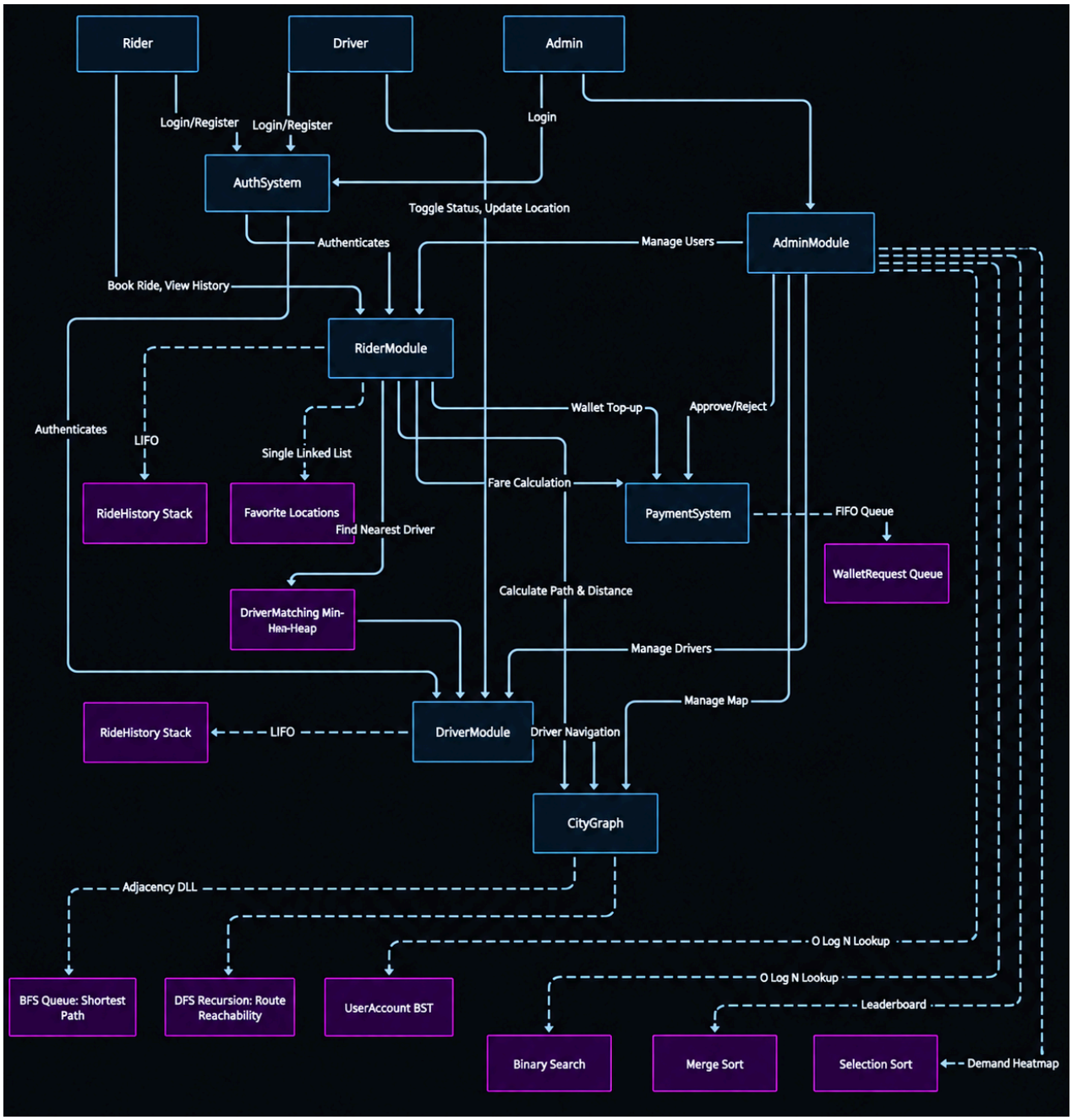
- *BFS:*

  This algorithm used in graph.cpp for calculating the shortest path between the passenger’s pickup spot and his destination. It replaces any complex algorithm like Dijkstra's algorithm .It’s calculates the shortest route accurately using queue,

- *DFS:*

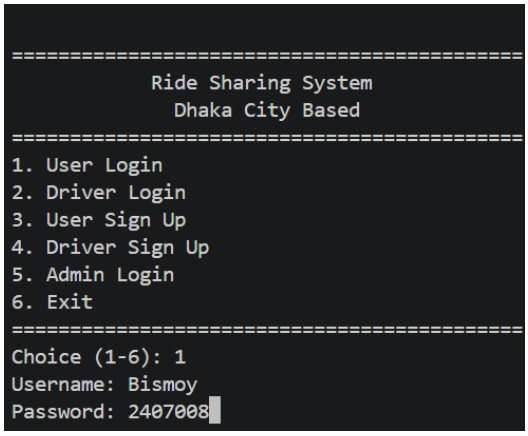
  This algorithm is used in graph.cpp for printing the alternative route along with the shortest path between passenger’s pickup and destination.It uses stack to display the places accordingly.

5.Flow Chart


Rider
Driver
Admin
Login/Register
Login/Register
Login
AuthSystem
Toggle Status, Update Location
Authenticates
Manage Users
AdminModule
Book Ride, View History
RiderModule
Authenticates
LIFO
Single Linked List
Wallet Top-up
Approve/Reject
Fare Calculation
RideHistory Stack
Favorite Locations
PaymentSystem
FIFO Queue
Find Nearest Driver
WalletRequest Queue
Calculate Path & Distance
DriverMatching Min-Hea-Heap
Manage Drivers
Manage Map
RideHistory Stack
LIFO
DriverModule
Driver Navigation
CityGraph
Adjacency DLL
O Log N Lookup
O Log N Lookup
Leaderboard
BFS Queue: Shortest Path
DFS Recursion: Route Reachability
UserAccount BST
Binary Search
Merge Sort
Selection Sort
Demand Heatmap

6.Interface overview

- *Main menu(Authentication):*



Here at first ,the person's identity is verified through unique username and password,which is already preserved in users.csv,drivers.csv file.It is performed through the **`AuthSystem`** class which interacts with both the `RiderModule` and `DriverModule` to verify credentials and assign user roles .The following code snippets provides the necessary functions

```
Role AuthSystem::lgn(const string &u, const string &p) const {
  if (u == "admin" && p == "123456")
    return ADMIN;

  if (rm) {
    RiderInfo *r = rm->gtrdr(u);
    if (r && r->password == p)
      return RIDER;
  }

  if (dm) {
    DriverInfo *d = dm->gtdr(u);
    if (d && d->password == p)
      return DRIVER;
  }

  return NONE;
}
```

```
bool AuthSystem::hsUsr(const string &u) const {
  if (u == "admin")
    return true;
  if (rm && rm->fndrdr(u) != -1)
    return true;
  if (dm && dm->fnddrv(u) != -1)
    return true;
  return false;
}
```

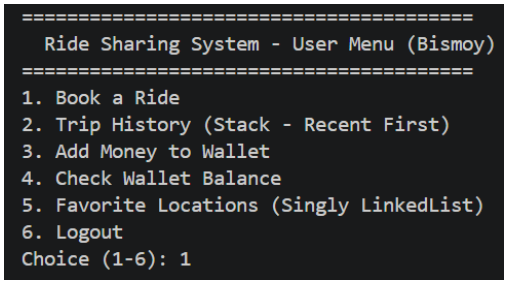
```
enum Role { RIDER, DRIVER, ADMIN, NONE };

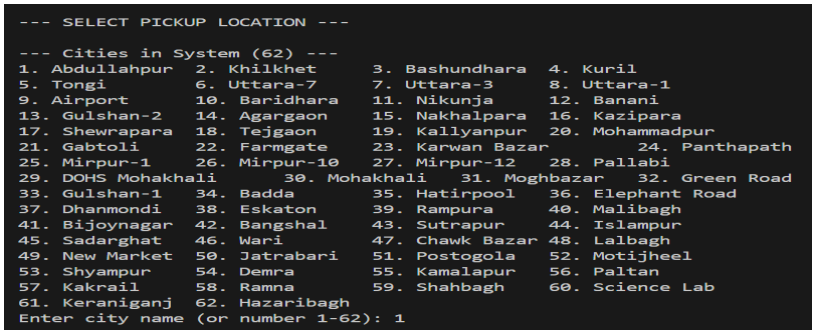
class AuthSystem {
private:
  RiderModule *rm;
  DriverModule *dm;

public:
  AuthSystem(RiderModule *r = nullptr, DriverModule *d = nullptr);

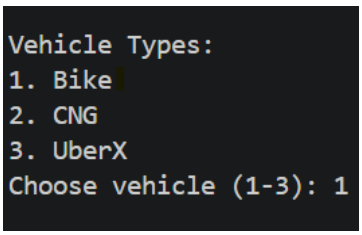

  Role lgn(const string &u, const string &p) const;
  bool hsUsr(const string &u) const;
};
```

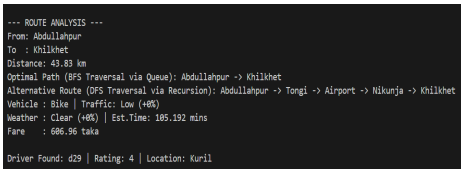
- *Book Riding(User interface)*
  In the user interface whenever the passenger selects book a ride,the system prints the location list . Then the passenger chooses his pickup and destination through indexing. Then he selects his vehicle type which determines the fare .Finally after confirmation a dynamic print is shown which prints his travelling path .Here are some snippets

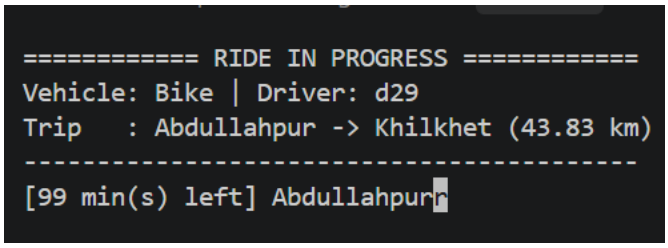


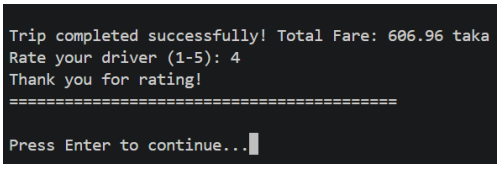












The whole process is done by using graph for calculating the nearest path using bfs and Mean heap to search the nearest driver in that location.The drivers are shown in Descending order based on their ratings (If tied then trip counts). Here are some code snippets

```
bool CityGraph::bfs(int start, int goal, Vector<int>& path, double& totalDist) {
    path.clr();
    totalDist = 0.0;
    int count = cities.sz();
    if (start < 0 || start >= count || goal < 0 || goal >= count) return false;
    if (start == goal) {
        path.pshbk(start);
        return true;
    }

    bool* visited = new bool[count]();
    int* parent = new int[count];
    double* edgeW = new double[count]();
    for (int i = 0; i < count; i++) parent[i] = -1;

    BFSQueue q;
    visited[start] = true;
    q.enq(start);

    bool reached = false;
    while (!q.emp()) {
        int curr = q.deq();
        if (curr == goal) {
            reached = true;
            break;
        }

        for (EdgeNode* e = adjList[curr]; e; e = e->next) {
            if (!visited[e->dest]) {
                visited[e->dest] = true;
                parent[e->dest] = curr;
                edgeW[e->dest] = e->weight;
                q.enq(e->dest);
            }
        }
    }

    if (reached) {
        Vector<int> tempPath;
        int curr = goal;
        while (curr != -1) {
            tempPath.pshbk(curr);
            if (curr != start) totalDist += edgeW[curr];
            curr = parent[curr];
        }
```

- *Trip History(User interface):*
  This feature allows the passenger to print his trip history in the console

```
Choice (1-6): 2

--- YOUR TRIP HISTORY (Most Recent First via Stack [LIFO]) ---
1. Sun Oct 04 02:53:03 2026 | Driver: d29 | Abdullahpur -> Khilkhet | Bike | Fare: 606.96 taka
2. Sat Oct 03 16:59:32 2026 | Driver: d9 | Mirpur-10 -> Karwan Bazar | Bike | Fare: 280.816 taka
3. Sat Oct 03 15:36:00 2026 | Driver: d1001 | Motijheel -> Gabtoli | Bike | Fare: 404.175 taka
4. Sat Oct 03 15:28:57 2026 | Driver: d47 | Green Road -> Wari | UberX | Fare: 577.167 taka

Press Enter to continue...
```

  This is implemented by customized stack which appends the data from rides.csv file.The stack is situated in stack.cpp here is some snippets

```
//user driver er
void shwHist(const string& username, bool isDriver) {
    RideHistoryStack st;
    if (!ldRds(st, username, isDriver) || st.emp()) {
        cout << (isDriver ? "No completed rides yet.\n" : "No trips yet.\n");
        return;
    }

    cout << (isDriver ? "\n--- YOUR COMPLETED RIDES (Most Recent First via Stack [LIFO]) ---\n"
                      : "\n--- YOUR TRIP HISTORY (Most Recent First via Stack [LIFO]) ---\n");
    int rank = 1;
    while (!st.emp()) {
        RideRecord r = st.pp();
        cout << rank++ << ". " << r.timestamp << " | "
             << (isDriver ? ("User: " + r.user) : ("Driver: " + r.driver)) << " | "
             << r.from << " -> " << r.to << " | " << r.vehicle
             << " | Fare: " << r.fare << " taka\n";
    }
}
```

```
Fri Oct 02 18:31:52 2026,Farhan,d37,Agargaon,Bangshal,11.7847,317.231,Bike
Fri Oct 02 18:32:52 2026,Farhan,d37,Bangshal,Uttara-7,19.8317,469.32,Bike
Fri Oct 02 18:41:46 2026,Avik,d565,Agargaon,Badda,5.22202,193.196,Bike
Sat Oct 03 00:53:50 2026,Farhan,d325,Pallabi,New Market,11.8785,199.785,Bike
Sat Oct 03 01:36:50 2026,Farhan,d29,Pallabi,Kuril,13.9245,220.245,Bike
```

- *Favourite locations(User interface):*

This feature allows the user to add ,show and delete his favourite locations. In this case, a singly linked list from linkedlist.cpp is used. It stores the favourite location in the user.csv file. And in case of adding and removing it just adds or deletes the node

```
case 5: {
  RiderInfo *r = rm.gtrdr(u);
  if (r) {
    cout << "1. View Favorites\n";
    cout << "2. Add Favorite Location\n";
    cout << "3. Remove Favorite Location\n";
    int fCh = inp("Choice (1-3): ", 1, 3);
    if (fCh == 2) {
      cg.shwcty();
      string fav = inp("Enter city name (or number 1-" + to_string(cg.cities.sz()) + "): ");
      int idx = -1;
      try {
        int num = stoi(fav);
        if (num >= 1 && num <= cg.cities.sz()) idx = num - 1;
      } catch (...) {}
      if (idx < 0) idx = cg.fndcty(fav);

      if (idx >= 0) {
        r->favorites.insrtTl(cg.cities[idx]);
        rm.sv("users.csv");
        cout << "Location '" << cg.cities[idx] << "' added to favorites (Singly LinkedList)!\n";
      } else {
        cout << "Invalid city name.\n";
      }
    } else if (fCh == 3) {
      if (r->favorites.emp()) {
        cout << "No favorite locations to remove.\n";
      } else {
        cout << "Favorites:\n";
        for (int i = 0; i < r->favorites.sz(); ++i) {
          cout << "  " << i + 1 << ". " << r->favorites.get(i) << "\n";
        }
        int removeChoice = inp("Choose favorite number to remove: ", 1, r->favorites.sz());
        string removed = r->favorites.get(removeChoice - 1);
        if (r->favorites.delNode(removed)) {
          rm.sv("users.csv");
          cout << "Location '" << removed << "' removed from favorites.\n";
        }
      }
    } else {
```

- *Driver interface:*

In this interface ,the employed drivers can see their profile and earnings,toggle his status,ssee his ride history and can update his current location .This interface is also similar to user interface as both of them almost use the same functions and modules here is the snippet of the interfaces

```
========================================
  Ride Sharing System - Driver Menu (d12)
  Status   : ONLINE
  Location: Farmgate
========================================
1. View Profile & Earnings
2. Toggle Online/Offline Status
3. Completed Rides History (Stack LIFO)
4. Update Current Location
5. Logout
Choice (1-5):
```

- *Admin interface:*

It is executed by AdminModule which handles vital systems totals including active riders,approval of top ups,identifying top demanding area,updating prices,printing logs and others. In short , the most versatile and efficient operations are done here in admin panel,

```
===== ADMIN MENU =====
1. Dashboard
2. Cities & Roads
3. Traffic Profile
4. Weather Profile
5. Users (Search/Delete)
6. Drivers (Search/Delete)
7. Pricing
8. Wallet Top-up Queue
9. Demand Heatmap
10. All Rides Log
11. Logout
Choice (1-11):
```

Cities & Roads (Map Management)

View Map	Handled by **AdminModule::viewMap()**. This function parses the adjacency connections from **connected_nodes.csv** and visually prints the entire road network structure to the console.

Add / Remove City	Executed by **AdminModule::addCity()** and **AdminModule::removeCity()**. These functions interact directly with the core **CityGraph** object to mutate the map nodes, subsequently updating the **connected_nodes.csv** data store.
Add / Remove Road	Executed by **AdminModule::addRoad()** and **AdminModule::removeRoad()**. These utilize the **selectTwoCities()** helper function to prompt and validate the edge connections (distances) between specific graph nodes before saving.

User (Rider) Management

Search User	Managed by **AdminModule::searchUserByBST()**. This leverages the Binary Search Tree algorithm via **UserAccountBST::search()** to quickly find a user with a time complexity of *O(log N)*.
Delete User	Handled by **AdminModule::removeUser()**, which locates the specific rider and completely removes the user from both the **RiderModule** data structure and the underlying **userBST** lookup tree.
List All Users	Executed by **AdminModule::listAllUsers()**, which subsequently triggers **RiderModule::displayAllRiders()** to print a formatted list of all registered passengers.

Driver Management

Search Driver	Handled by **AdminModule::searchDriverByRBS()**. It queries the system by calling **DriverModule::getDriverByID()** and utilizes the **binarySearchRecursive()** algorithm for rapid execution.
Driver Leaderboard	Generated within **AdminModule::handleDriverMenu()**, which calls **DriverModule::getLeaderboard()**. This returns the top drivers sorted by their rating and earnings using the robust Merge Sort algorithm.
Delete Driver	Executed by **AdminModule::removeDriver()**. The process locates the driver via **DriverModule::getDriver()** and purges them completely from the system's runtime and storage.

List All Drivers	ecuted by **AdminModule::listAllDrivers()**, which iterates through the storage vector by invoking **DriverModule::displayAllDrivers()**.

Wallet & Financial Systems

View Wallet Queue	Triggered by **AdminModule::handleWalletMenu()**, which invokes **AdminModule::viewWalletRequests()** to display a list of all pending fiat-to-digital balance top-up requests.
Approve / Reject	Processed directly within **AdminModule::handleWalletMenu()** by invoking **PaymentSystem::approveFirstRequest()** to securely credit the user's wallet, or **PaymentSystem::rejectFirstRequest()** to deny the transaction and clear the queue item.
Pricing Config	Handled by **AdminModule::handlePricingMenu()**. This interface allows the admin to update variables like **pm.demandWeight**, **pm.maxSurgeCap**, and individual vehicle base fares, immediately saving the results to **pricing_config.csv**.

Data Visualization & Profiles

Demand Heatmap	Executed by **AdminModule::showDemandHeatmap()**. This feature parses **rides.csv** and utilizes **getTopDemandSelectionSort()** to rank and display cities strictly by ride request volume.
All Rides Log	Handled by **AdminModule::viewSortedRides()**, which loads and displays the global historical trip data across the entire platform.
Traffic & Weather	Managed by **AdminModule::editTrafficProfile()** and **AdminModule::editWeatherProfile()**. Both functions route into the generic helper **editGenericProfile()** to modify and store dynamic surge multipliers for specific hours of the day.

Here some code snippets are added

```
UserBSTNode* UserAccountBST::insrthlpr(UserBSTNode* node, int id, const std::string& username) {
    if (!node) {
        return new UserBSTNode(id, username);
    }
    if (id < node->id) {
        node->left = insrthlpr(node->left, id, username);
    } else if (id > node->id) {
        node->right = insrthlpr(node->right, id, username);
    } else {
        node->username = username;
    }
    return node;
}

UserBSTNode* UserAccountBST::srchhlpr(UserBSTNode* node, int id) const {
    if (!node || node->id == id) return node;
    if (id < node->id) return srchhlpr(node->left, id);
    return srchhlpr(node->right, id);
}

UserBSTNode* UserAccountBST::fndminnd(UserBSTNode* node) const {
    while (node && node->left) node = node->left;
    return node;
}

UserBSTNode* UserAccountBST::delhlpr(UserBSTNode* node, int id, bool& deleted) {
    if (!node) {
        deleted = false;
        return nullptr;
    }
    if (id < node->id) {
        node->left = delhlpr(node->left, id, deleted);
    } else if (id > node->id) {
        node->right = delhlpr(node->right, id, deleted);
    } else {
        deleted = true;
        if (!node->left) {
            UserBSTNode* temp = node->right;
            delete node;
            return temp;
        } else if (!node->right) {
            UserBSTNode* temp = node->left;
            delete node;
            return temp;
        }
        UserBSTNode* succ = fndminnd(node->right);
        node->id = succ->id;
        node->username = succ->username;
        bool dummy = false;
        node->right = delhlpr(node->right, succ->id, dummy);
    }
    return node;
}

void UserAccountBST::dstryhlpr(UserBSTNode* node) {
    if (!node) return;
    dstryhlpr(node->left);
    dstryhlpr(node->right);
    delete node;
}
```

```
void AdminModule::bldBST(RiderModule &rm) {
  for (int i = 0; i < rm.riders.sz(); i++) {
    userBST.insrt(rm.riders[i].id, rm.riders[i].username);
  }
}

void AdminModule::insrtBST(int id, const string& username) {
  userBST.insrt(id, username);
}

void AdminModule::fndBST(RiderModule &rm, int id) {
  string *res = userBST.srch(id);
  RiderInfo *r = res ? rm.gtrdr(*res) : nullptr;
  if (r) {
    cout << "[BST Search] ID: " << r->id << " | User: " << r->username
         << " | Wallet: " << r->wallet << " taka\n";
  } else {
    cout << "User with ID " << id << " not found.\n";
  }
}
```

```
// Merge Sort Driver Leaderboard er jonno
static void mrgdrv(int* arr, int left, int mid, int right, const DriverInfo* drivers) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int* leftArr = new int[n1];
    int* rightArr = new int[n2];

    for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++) rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (better(drivers[leftArr[i]], drivers[rightArr[j]])) {
            arr[k++] = leftArr[i++];
        } else {
            arr[k++] = rightArr[j++];
        }
    }
    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];

    delete[] leftArr;
    delete[] rightArr;
}

void mrgesrt(int* arr, int left, int right, const DriverInfo* drivers) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mrgesrt(arr, left, mid, drivers);
    mrgesrt(arr, mid + 1, right, drivers);
    mrgdrv(arr, left, mid, right, drivers);
}
```

```
int BnrSrchRcrsn(const DriverInfo* arr, int low, int high, int targetId) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid].id == targetId) return mid;
    if (arr[mid].id > targetId)
        return BnrSrchRcrsn(arr, low, mid - 1, targetId);
    return BnrSrchRcrsn(arr, mid + 1, high, targetId);
}
```

```cpp
// Selection Sort to sort view demand heatmap e use hoise
void selsort(Vector<DemandItem>& items) {
    int n = items.sz();
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (items[j].count > items[maxIdx].count) {
                maxIdx = j;
            }
        }
        if (maxIdx != i) {
            swap(items[i], items[maxIdx]);
        }
    }
}
```

```cpp
void WalletRequestQueue::enq(const WalletRequest& data) {
    WalletQueueNode* newNode = new WalletQueueNode(data);
    if (emp()) {
        frontNode = rearNode = newNode;
    } else {
        rearNode->next = newNode;
        newNode->prev = rearNode;
        rearNode = newNode;
    }
    count++;
}

WalletRequest WalletRequestQueue::deq() {
    if (!frontNode) return WalletRequest();
    WalletQueueNode* temp = frontNode;
    WalletRequest val = temp->data;
    frontNode = frontNode->next;
    if (frontNode) {
        frontNode->prev = nullptr;
    } else {
        rearNode = nullptr;
    }
    delete temp;
    count--;
    return val;
}

bool WalletRequestQueue::emp() const {
    return frontNode == nullptr;
}

int WalletRequestQueue::gtsz() const {
    return count;
}

WalletQueueNode* WalletRequestQueue::frntnd() const {
    return frontNode;
}

void WalletRequestQueue::clr() {
    while (!emp()) {
        deq();
    }
}
```

6.Discussion

Custom Data Structure Implementation

The project prioritizes optimized manual memory management by building fundamental data structures from scratch rather than defaulting to standard libraries.

- **Spatial Routing (Graphs):** The city network is mapped using a CityGraph adjacency list. This allows the application to execute Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to validate paths and compute travel distances between nodes.
- **Fast Retrieval (Trees & Heaps):** A Binary Search Tree (BST) architecture executes user lookups in $O(\log N)$ time. Driver matching is accelerated using a custom Min-Heap (DriverMatchingMinHeap), which instantly extracts the most optimal available driver for a new ride request.
- **Sequential Processing (Stacks & Queues):** Financial transactions, such as wallet top-ups, are managed equitably through a Doubly Linked List-backed FIFO Queue. Conversely, individual trip logs enforce chronological integrity by pushing and popping records via a LIFO Stack, ensuring the most recent rides are always accessed first.

Algorithmic Efficiency

Administrative sorting and searching tasks are paired with algorithms suited to their specific computational constraints. Merge Sort ($O(N \log N)$) handles the driver leaderboard, providing a stable, high-performance sort as the driver pool scales. Selection Sort is applied to generate the demand heatmap, offering a low-overhead solution for smaller, localized arrays. Recursive Binary Search is utilized to parse the driver arrays, cutting down search times significantly compared to linear iteration.

Data Persistence

The application maintains absolute state consistency across execution cycles through flat-file storage. Data is serialized and deserialized into CSV formats (rides.csv, connected_nodes.csv, pricing_config.csv). Complex nested data, such as a rider's SinglyLinkedList of favorite locations, is successfully preserved using specialized delimiters (semicolons) alongside standard comma separation.

The integration of these custom data structures and OOP principles results in a cohesive simulation of a high-throughput ride-hailing backend. The application strictly enforces algorithmic efficiency, mirroring the low-latency requirements of real-world transactional systems. The dynamic pricing

engine—which calculates fares by parsing concurrent weather profiles, traffic conditions, and peak demand weights—adds practical economic logic to the software's capabilities.

Because the backend logic is highly modular and independent of the user interface, the existing C++ framework provides a direct pathway for future expansion. The current flat-file persistence layer can be smoothly swapped for a relational database, while the core routing and matching engines are well-structured for integration into a graphical interface, network-based API, or real-time coordinate tracking system.